

MODULE 5 · STORAGE

PersistentVolumes & PVCs

Storage as a cluster resource, and a claim against it

Storage that outlives the Pod

The volumes from Lab 5.1 die with the Pod. A **PersistentVolume (PV)** is a piece of storage provisioned in the cluster — its lifecycle is independent of any Pod.

A **PersistentVolumeClaim (PVC)** is how a Pod actually gets one: a request for storage that Kubernetes **binds** to a matching PV. Once bound, the PVC is a stable handle — delete and recreate the Pod, and the data underneath is untouched.

BY THE END YOU CAN

- Explain the PV vs PVC split
- Bind a PVC to a PV statically
- Mount a PVC as a Pod volume
- Prove data survives a Pod restart

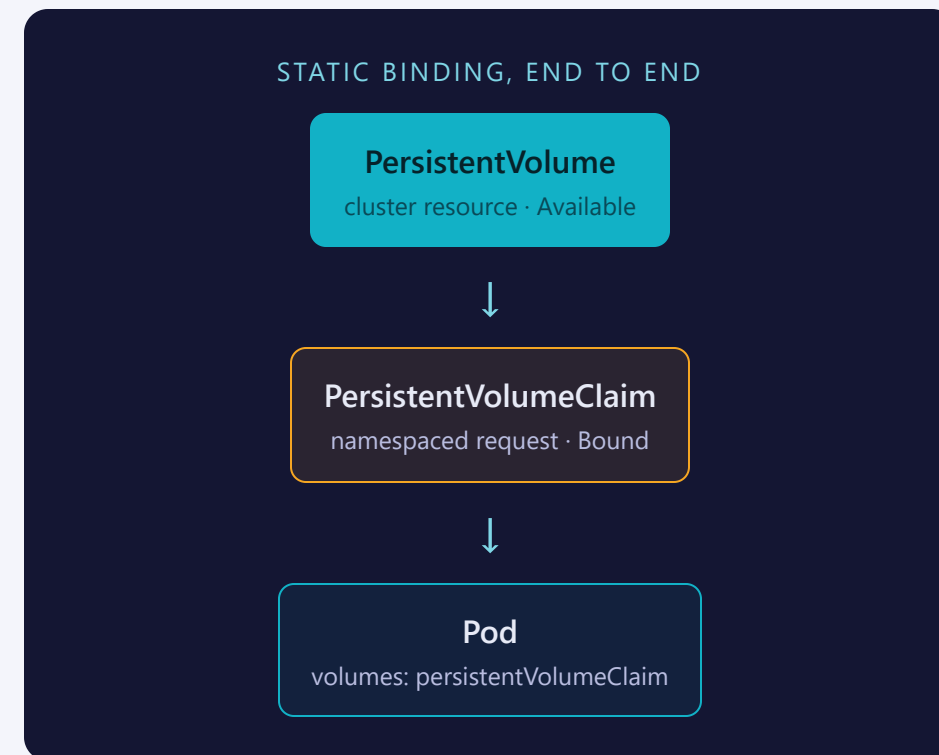
CONCEPT

Supply and demand for storage

Two objects, two owners:

- **PersistentVolume** — a cluster-scoped resource. Someone (an admin, or a provisioner) says "this storage exists."
- **PersistentVolumeClaim** — a namespaced request. A team says "I need storage like this," without naming a disk.

Kubernetes matches the two, marks them **Bound**, and only then will a Pod mount the PVC.



Same storage, two different vantage points

PERSISTENTVOLUME (PV)	PERSISTENTVOLUMECLAIM (PVC)
Cluster-scoped — no namespace	Namespaced, just like the Pod that uses it
Created by an admin or a provisioner	Created by a user / app team
Advertises capacity, access modes, reclaim policy	Requests capacity and access modes
<code>storageClassName: ""</code> opts out of dynamic provisioning	<code>storageClassName: ""</code> must be set to bind statically

THE STATIC-BINDING RULE

To bind a PVC to a hand-made PV instead of triggering dynamic provisioning, **both** objects need `storageClassName: ""` — set on the PV and matched exactly on the PVC. Leave it unset on the PVC and a default StorageClass may take over instead (Lab 5.3).

The PersistentVolume — storage the cluster owns

pv.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: lab-pv
  labels:
    type: local
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ""
  hostPath:
    path: /tmp/lab-pv-data
```

storageClassName: "" opts this PV out of dynamic provisioning, keeping it free for a static bind.

The PersistentVolumeClaim — a request that matches it

pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: lab-pvc
  namespace: training
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: ""
  volumeName: lab-pv
```

storageClassName: "" and volumeName together force a static bind to lab-pv instead of dynamic provisioning.

There's no shortcut generator here

ONE-SHOT — KUBECTL CREATE -F

```
kubectl create -f pv.yaml
```

Submits it once; fails if it already exists

DECLARATIVE — KUBECTL APPLY -F

```
kubectl apply -f pv.yaml
```

Upserts and diffs against the last-applied config

NO KUBECTL RUN HERE

Unlike Pods or Deployments, PVs and PVCs have no imperative generator — you always author YAML. You can still preview it before committing:

```
$ kubectl apply -f pvc.yaml \
  --dry-run=client -o yaml
```

SEE IT

A Pod that mounts the bound PVC

pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: pvc-pod
  namespace: training
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - /bin/sh
        - -c
        - "echo 'Persistent data written at: '$(date) > /data/output.txt && sleep 3600"
      volumeMounts:
        - name: persistent-storage
          mountPath: /data
  volumes:
    - name: persistent-storage
      persistentVolumeClaim:
        claimName: lab-pvc
```

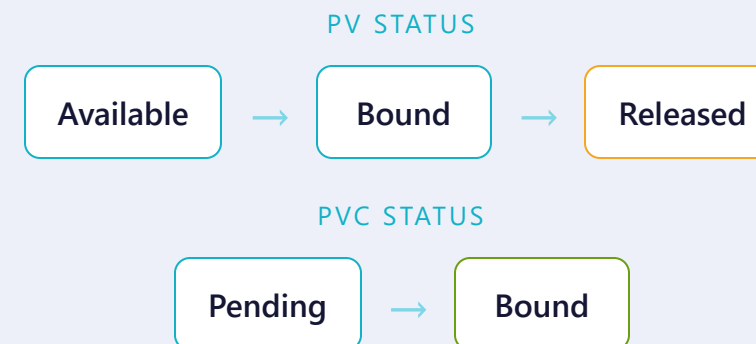
The Pod never names lab-pv — only lab-pvc, via `persistentVolumeClaim.claimName`.

SEE IT

Delete the Pod, keep the data

```
$ kubectl exec pvc-pod -n training -- cat /data/output.txt
Persistent data written at: Mon Jul 6 09:14:02 UTC

$ kubectl delete pod pvc-pod -n training --force --grace-period=0
$ kubectl apply -f pod.yaml
$ kubectl exec pvc-pod -n training -- cat /data/output.txt
Persistent data written at: Mon Jul 6 09:14:02 UTC # same line
```



Reclaim policy decides what happens next

POLICY	WHEN THE PVC IS DELETED
Retain	PV keeps its data, status becomes Released — needs manual cleanup before reuse
Delete	PV and its backing storage are deleted automatically
Recycle	Deprecated — a basic <code>rm -rf</code> of the volume, avoid it

FREEING A RELEASED PV

lab-pv uses Retain. Deleting lab-pvc leaves it Released, not Available — it refuses new claims until you strip the old binding:

```
$ kubectl patch pv lab-pv --type json \
  -p ' [{"op": "remove", "path": "/spec/claimRef"}]'
```

Where people trip

- **storageClassName mismatch.** Leave it unset on the PVC and it ignores your hand-made PV — or waits forever if no default StorageClass exists.
- **Namespace confusion.** A PV has none; its PVC does. `kubectl get pvc` without `-n` looks in the wrong place.
- **Capacity or access-mode mismatch.** A PVC asking for more than the PV offers, or a mode it doesn't support, never binds — it just sits Pending.
- **Released ≠ Available.** Under Retain, deleting the PVC does not free the PV — `spec.claimRef` must be cleared by hand.

Commands to keep at hand

```
$ kubectl get pv                # cluster-wide, no namespace
$ kubectl get pvc -n training    # namespaced
$ kubectl describe pv lab-pv     # Claim field = bound PVC
$ kubectl describe pvc lab-pvc -n training # Volume field = bound PV
$ kubectl exec POD -n training -- cat FILE # peek at the mounted data
$ kubectl patch pv NAME --type json -p '[...remove claimRef]'
```

Tip: a PVC stuck Pending — describe its Events before touching the PV.

RECAP

PVs and PVCs in one breath

- PV = cluster storage resource; PVC = a namespaced request for it
- Static binding needs `storageClassName: ""`, set and matched on both
- A Pod mounts storage through a PVC, never a PV directly
- Reclaim policy decides what happens to data after the PVC is deleted

HANDS-ON

Now run Lab 5.2
[labs/5.2/](#)

Next: [5.3 — StorageClasses & Dynamic Provisioning](#)